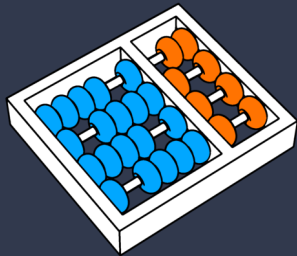


UNICAMP



# Avaliação II

Prof. Dr. André Gomes Regino

regino@unicamp.br

# Agenda

**01.**O Problema: LLMs em produção vs. em notebook

**02.**Componentes de um sistema real com LLM

**03.**Observabilidade: Logs, Métricas e Tracing

**04.**Monitoramento Contínuo e Drift

**05.**Avaliação de Sistemas RAG

**06.**Avaliação Online (feedback de usuários)

**07.**Checklist de Produção e Ferramentas

# O Problema

**O modelo funcionou no notebook...** mas falhou em produção.

Abaixo estão exemplos comuns de falhas que ocorrem no mundo real.



## Respostas lentas

Latência que impacta a experiência do usuário final.



## Alucinações ocasionais

Geração de informações falsas ou sem sentido.



## Documentos irrelevantes

Falhas no processo de recuperação (RAG).



## Aumento de custo

Consumo inesperado de tokens em escala.



## Inconsistência

Respostas variando sem alteração no input.



## Prompts frágeis

Pequenas mudanças quebram o comportamento.

### Ideia central

Avaliar um LLM isoladamente não é suficiente. **É preciso avaliar o sistema como um todo.**

# O Problema

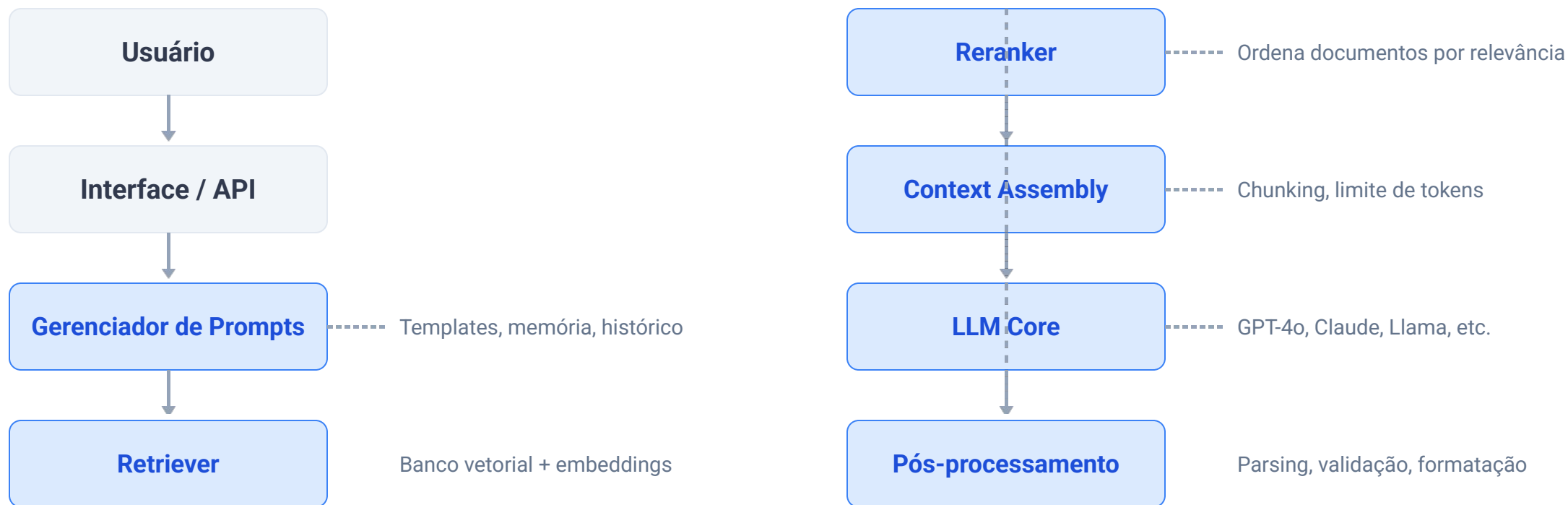
## Por que isso acontece?

A transição do desenvolvimento para o ambiente real traz novos desafios.

| NOTEBOOK             | VERSUS | PRODUÇÃO                   |
|----------------------|--------|----------------------------|
| 1 prompt fixo        | ↔      | milhares de variações      |
| 1 documento de teste | ↔      | base vetorial dinâmica     |
| sem usuários reais   | ↔      | comportamentos imprevistos |
| sem latência crítica | ↔      | SLA de 2s                  |
| sem custo acumulado  | ↔      | \$X mil/mês                |

# Sistemas com LLM em Produção

Um sistema real com LLM possui múltiplos componentes conectados:



Cada seta é um **ponto de falha potencial**. Uma resposta ruim pode vir de qualquer componente.

# Objetivos

1. **Entender** sobre logs estruturados para sistemas com LLM
2. **Entender** e aplicar tracing distribuído em pipelines com múltiplos componentes (retriever → LLM → resposta)
3. **Calcular** e interpretar Precision@k, Recall@k e Groundedness
4. **Identificar** as principais falhas em pipelines RAG
5. **Estruturar** um checklist de monitoramento contínuo para sistemas com LLM em produção

# Observabilidade

# O que é Observabilidade

## Capacidade de entender:

- O que o usuário perguntou exatamente?
- Qual prompt foi enviado (após templates)?
- Quais documentos foram recuperados?
- Quanto tempo cada etapa levou?
- Quantos tokens foram consumidos?
- Onde exatamente ocorreu uma falha?

## Componentes Observáveis:



## Sem observabilidade:

*"O sistema deu uma resposta errada"*

→ Causa desconhecida (retriever, prompt ou LLM?)

## Com observabilidade:

*"Retriever trouxe doc sobre México vs pergunta sobre Canadá"*

→ **Identificado:** falha no embedding/chunking.



# Três Pilares da Observabilidade

## Logs

### O que é:

Registro discreto de eventos com timestamp.

### Exemplo em LLMs:

- Prompt e resposta
  - Tokens e latência
  - User ID
- "o que aconteceu"

## Metrics

### O que é:

Valores numéricos agregados ao longo do tempo.

### Exemplo em LLMs:

- Latência p95
  - Custo/dia e taxa de erro
  - Tokens por requisição
- "frequência / intensidade"

## Tracing

### O que é:

Rastreo do fluxo completo de uma requisição.

### Exemplo em LLMs:

user → retriever → reranker → LLM →  
resposta  
→ "onde no fluxo aconteceu"

Os três pilares são complementares para uma visão holística do sistema.

# Logs



## Identificação

- timestamp
- user\_id
- session\_id



## Conteúdo

- prompt (template + variáveis)
- resposta gerada
- documentos (IDs + scores)



## Performance

- latência por etapa
- tokens (in/out)
- custo estimado



## Configurações

- modelo e versão
- temperatura
- outros parâmetros



## Monitoramento

- erros e exceções
- avisos (warnings)

## Campos essenciais para Logs de LLM

Estes dados permitem depurar alucinações e otimizar custos.

# Exemplo de Log em Python

```
# pip install structlog==24.4.0
import structlog, time

log = structlog.get_logger()

def chamar_llm(prompt: str, user_id: str) -> dict:
    t0 = time.time()
    # ... chamada ao modelo ...
    latencia = time.time() - t0

    log.info("llm_call",
            user_id=user_id,
            input_tokens=820,
            output_tokens=230,
            latency_s=round(latencia, 3),
            model="gpt-4o-2024-11-20",
            cost_usd=round((820*2.5 + 230*10) / 1_000_000, 5)
    )
```

O structlog permite gerar logs estruturados (JSON), facilitando a análise de métricas como latência e custo.

# Tracing

Trace: **pergunta do usuário** → **resposta final**

**Span 1: retrieval** [0ms – 312ms]

↳ embed\_query: 45ms ↳ vector\_search: 267ms

**Span 2: reranking**

[312ms – 400ms] • 88ms

**Span 3: prompt\_assembly**

[400ms – 430ms] • 30ms

**Span 4: llm\_call** [430ms – 4231ms]

🕒 TTFT: 890ms 🔄 Total Tokens: 1050 ⚡ Total Duration: 3801ms

**Span 5: post\_processing**

[4231ms – 4260ms] • 29ms

# Exemplo de Tracing

Pergunta: "Qual é a capital do Canadá?"

## retrieval

top\_k: 5 | query\_embedding: [0.21, -0.43, ...]

- doc\_041 (0.82) → "cidades dos EUA" ⚠
- doc\_072 (0.79) → "capitais da América"
- doc\_015 (0.71) → "turismo no México" ⚠

## llm\_call

Contexto: Docs sobre EUA e México

Resposta: "A capital do Canadá é Toronto." ✗

## Diagnóstico via tracing:

- Falha no **retriever**, não no LLM. O modelo respondeu com base no contexto errado.
- **Ação:** Revisar chunking e embeddings sobre América do Norte.

Sem tracing: "o modelo errou" | **Com tracing: "o retriever trouxe documentos errados"**

# Monitoramento Contínuo

## Sistemas mudam constantemente:

### Dados e Usuários

- Novos usuários com perfis diferentes
- Novos documentos na base vetorial
- Drift de dados (perguntas mudam)

### Modelos e Configurações

- Prompts ajustados pela equipe
- Atualização de modelo pelo provedor
- Mudanças em chunks e embeddings

### Degradação Silenciosa

Fique atento: mudanças sutis podem afetar drasticamente a qualidade sem disparar erros técnicos imediatos.

## Mudança no Sistema & Impacto Possível

Nova versão do modelo API

→ **Estilo de resposta muda**

Re-indexação dos documentos

→ **Retrieval score cai**

Aumento do volume de queries

→ **Latência sobe (p95)**

Novos tópicos de perguntas

→ **Groundedness cai**

# O que Monitorar?

**SLI** (Service Level Indicator) → valor medido **SLO** (Service Level Objective) → meta aceitável

| Indicador (SLI)                                                                                     | O que mede           | SLO típico |
|-----------------------------------------------------------------------------------------------------|----------------------|------------|
|  Latência p95      | Tempo de resposta    | < 3s       |
|  Throughput        | Requisições/segundo  | > 10 req/s |
|  Taxa de erro      | Falhas / total req.  | < 1%       |
|  Custo / 1k req.   | Gasto em tokens      | < \$0.50   |
|  Groundedness    | Resposta vs contexto | > 0.75     |
|  Precision@5     | Docs relevantes top5 | > 0.70     |
|  Satisfação      | Feedback do usuário  | > 80%      |
|  Alucinação rate | Respostas inventadas | < 5%       |

# Avaliação de Sistemas RAG



# O Pipeline Precisa ser Avaliado

## Componentes Avaliáveis



Chunking



Embeddings



Retrieval



Reranking



Contexto Enviado



Resposta Final

⚠ Uma resposta ruim pode surgir mesmo com um ótimo LLM.

# Precision@k



## Ideia Central: Precisão na Recuperação

Mede a proporção de documentos úteis entre os resultados retornados.

| Rank | Documento Retornado                 | Relevante? |
|------|-------------------------------------|------------|
| 1    | "Algoritmo de backpropagation"      | ✓          |
| 2    | "História das redes neurais"        | ✗          |
| 3    | "Gradiente descendente e derivadas" | ✓          |
| 4    | "Cálculo de gradientes em MLPs"     | ✓          |
| 5    | "Introdução ao Python"              | ✗          |

### Cálculo do Resultado

Precision@5

$$3 / 5 = 0.60$$

Precision@3

$$2 / 3 = 0.67$$

# Recall@k



**Pergunta: Dos documentos relevantes existentes, quantos foram encontrados?**

Fórmula:  $\text{Recall@k} = (\text{relevantes encontrados nos top-k}) / (\text{total relevantes})$

| k  | Relevantes top-k | Recall@k |
|----|------------------|----------|
| 5  | 3 relevantes     | 0.30     |
| 10 | 6 relevantes     | 0.60     |
| 20 | 9 relevantes     | 0.90     |

## Exemplo: "Backpropagation"

Total de docs relevantes na base: 10

Recall@5

$$3 / 10 = 0.30$$



Aumentar k melhora o Recall, mas introduz mais ruído no processamento.

↗ Mais documentos recuperados = maior recall, mas maior custo computacional.

# Trade-off

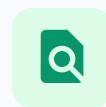


## Alta Precision, Baixo Recall

### Retriever conservador

- Só traz o que tem certeza.
- Poucos documentos irrelevantes.
- Pode perder informações importantes.

**Risco: Resposta incompleta**



## Alto Recall, Baixa Precision

### Retriever abrangente

- Traz muitos documentos para a LLM.
- Maior chance de encontrar tudo.
- Introduz muito ruído (irrelevância).

**Risco: LLM confuso ou caro**

**⚠ Problema:** Melhorar um pode piorar o outro. Encontrar o equilíbrio é a chave.

# Groundedness



## A resposta está fundamentada?

A resposta realmente se apoia no contexto recuperado?



## Definição

Groundedness mede em que grau cada afirmação da resposta pode ser verificada nos documentos recuperados.



## O modelo falha quando:

- Inventa informações (alucinação)
- Mistura conhecimento interno não solicitado
- Extrapola além do contexto fornecido

# Exemplo de Groundedness

```
example.py

# pip install ragas==0.2.15
from ragas import evaluate
from ragas.metrics import faithfulness # faithfulness ≈ groundedness
from datasets import Dataset

data = Dataset.from_dict({
    "question": ["Qual a garantia do produto?"],
    "answer": ["O produto possui garantia vitalícia."],
    "contexts": [["O produto possui garantia de 1 ano."]],
})

result = evaluate(data, metrics=[faithfulness])
print(result["faithfulness"]) # → 0.0 (contradição direta)
```

<> O framework RAGAS permite avaliar se a resposta é fiel ao contexto fornecido.

# Principais Falhas em RAG

| Falha                      | Sintoma                                          | Como detectar/corrigir                    |
|----------------------------|--------------------------------------------------|-------------------------------------------|
| Retrieval ruim             | Docs irrelevantes top-k                          | Medir Precision@k<br>Revisar embeddings   |
| Chunking inadequado        | Informação cortada no meio; contexto sem sentido | Checar coerência do chunk manualmente     |
| Contexto excessivo (ruído) | LLM ignora parte do contexto; resposta vaga      | Testar com k menor;<br>Adicionar reranker |
| Contexto insuficiente      | Resposta incompleta; alucinação para completar   | Aumentar k;<br>Usar expansão de query     |
| Alucinação do LLM          | Informação inventada, confiante e errada         | Medir Groundedness;<br>LLM-as-judge       |



Regra prática: Isole cada componente e teste separadamente antes de culpar o LLM.

# Como Detectar Alucinações

● ● ● detect\_hallucination.py

```
# Estratégia 1 – Comparação direta com o contexto (NLI)
# pip install transformers==4.51.3 torch

from transformers import pipeline

nli = pipeline("zero-shot-classification",
               model="cross-encoder/nli-deberta-v3-small")

contexto = "O produto possui garantia de 1 ano."
resposta = "O produto possui garantia vitalícia."

resultado = nli(resposta,
                candidate_labels=["entailment", "contradiction", "neutral"],
                hypothesis_template="{ }",
                multi_label=False)

# → contradiction: 0.91 ← alucinação detectada
```



# Como Detectar Alucinações

```
judge_strategy.py

# Estratégia 2 - LLM-as-a-Judge para factualidade
prompt = f"""Com base APENAS no contexto abaixo, a afirmação é verdadeira?
Contexto: {contexto}
Afirmação: {resposta}
Responda: VERDADEIRO / FALSO / NÃO MENCIONADO + justificativa."""

# Estratégia 3 - Regras automáticas (Verificação de Entidades)
def check_entities(resposta, documentos):
    # Verifica se datas, valores (R$ 5.000) e nomes existem nos docs
    if entity not in documentos:
        return "ALERTA: Alucinação detectada"

# Estratégia 4 - RAGAS Faithfulness
score = faithfulness.compute(resposta, contexto)
```

# Avaliação Online

## O Usuário como Fonte de Avaliação

### Sinais implícitos (sem esforço do usuário)

| Sinal          | Interpretação                 |
|----------------|-------------------------------|
| Reformulação   | Resposta não foi útil         |
| Abandono       | Frustração / Desinteresse     |
| Tempo leitura  | Resposta relevante (positivo) |
| Click em fonte | Verificou informação          |
| Cópia de texto | Resposta usada (positivo)     |

### Sinais explícitos (requerem ação)

- 👍 / 👎 (like/dislike) – baixa taxa de resposta
- Avaliação 1–5 estrelas
- Campo de texto livre ("O que estava errado?")
- Relatório de problema ("Marcar incorreto")

### Boas práticas

- **Thumbs down** é mais valioso que up
- Nunca pergunte mais de 1 coisa por feedback
- Amostragem: pedir em ~5–10% das sessões

# Drift

Tipos de drift em sistemas com LLM:

| Tipo de Drift                       | Exemplo concreto                                                                    |
|-------------------------------------|-------------------------------------------------------------------------------------|
| <b>Data drift</b><br>(distribuição) | Usuários começam a perguntar sobre tópicos não cobertos pelos documentos existentes |
| <b>Concept drift</b>                | O significado de "urgente" muda no contexto do negócio ao longo do tempo            |
| <b>Model drift</b>                  | Provedor atualiza silenciosamente o modelo (ex: gpt-4o → versão nova sem aviso)     |
| <b>Document drift</b>               | Re-indexação introduz chunks diferentes → Recall cai sem nenhuma mudança no código  |

# Checklist em Produção: O que Monitorar

## MODELO

- ☐ **Qualidade respostas**  
(Amostra diária LLM-judge)
- ☐ **Custo total**  
(Tokens × preço p/ dia)
- ☐ **Taxa truncagem**  
(out == max\_tokens)
- ☐ **Versão fixada**  
(Model string c/ data)

## RAG

- ☐ **Precision@k**  
(Sample do retriever)
- ☐ **Groundedness**  
(Alerta se < 0.70)
- ☐ **Contexto vazio**  
(Retriever falhou)
- ☐ **Scores similaridade**  
(Drift de embedding)

## INFRA

- ☐ **Latência p99**  
(Por endpoint)
- ☐ **Throughput**  
(Req/s e timeouts)
- ☐ **Memória / GPU**  
(Modelos locais)
- ☐ **Backpressure**  
(Fila de requisições)

## SEGURANÇA

- ☐ **Prompt Injection**  
(Regex + Classifier)
- ☐ **Vazamento PII**  
(Dados sensíveis)
- ☐ **Fora de escopo**  
(Tópicos proibidos)

## USUÁRIO

- ☐ **Thumbs down**  
(Alerta se > 15%)
- ☐ **Reformulação**  
(> 30% = sinal ruim)
- ☐ **Tempo sessão**  
(Taxa de abandono)

# Ferramentas Comuns

| Ferramenta       | Logs | Tracing | Dashboard | RAG Eval | Open Source |
|------------------|------|---------|-----------|----------|-------------|
| LangSmith        | ✓    | ✓       | ✓         | ✓        | Não         |
| LangFuse         | ✓    | ✓       | ✓         | ✓        | ✓           |
| Helicone         | ✓    | ✗       | ✓         | ✗        | ✓           |
| Weights & Biases | ✓    | ✓       | ✓         | ✓        | Não         |
| Arize Phoenix    | ✓    | ✓       | ✓         | ✓        | ✓           |
| OpenTelemetry    | ✓    | ✓       | ✗         | ✗        | ✓           |

# Ferramentas Comuns: LangFuse

```
# Exemplo rápido - LangFuse (maio 2025)
pip install langfuse==3.9.0
from langfuse import Langfuse

lf = Langfuse()
trace = lf.trace(name="rag_pipeline", user_id="usr_183")
span = trace.span(name="retrieval", input={"query": "..."})
span.end(output={"docs": ["doc_12", "doc_91"]},
metadata={"latency_ms": 312, "precision_at_5": 0.60})

# → disponível em dashboard em tempo real
```

## Recomendação

- Começar com **LangFuse** (open source, fácil de subir com Docker)
- Ou **LangSmith** se já usa LangChain / LangGraph

## Por que usar?

Ideal para monitoramento de latência e precisão em tempo real de pipelines RAG.

# Take Home Message

## 01. Avaliar o LLM isoladamente não é suficiente

O problema pode estar no retriever, no chunking ou no prompt — não no modelo.

## 02. Observabilidade é a base de tudo

Logs + Métricas + Tracing. Sem visibilidade, falhas são invisíveis até virar crise.

## 03. Precision@k e Recall@k

Medem o retriever, não o LLM. Um retriever ruim contamina qualquer resposta, mesmo com GPT-4o.

## 04. Groundedness no RAG

É a métrica central. Se não há fundamentação no contexto, o sistema não é confiável em produção.

## 05. Drift Silencioso

Monitore continuamente. Qualidade boa hoje não garante qualidade boa em 30 dias.

## 06. Thumbs Down

O usuário detecta o que as métricas não capturam. É o sinal mais honesto que existe.